

ThreadT ConfusionBuffer Anki

Thread T — Confusion Buffer & Anki Pack (DSA + C++)

Companion to the Threads section (runs continuously from the Phase-1 self-test, ~Week 10, in the first ~45 min of Block B). ~275 hrs over the program. This is the deck that keeps every coding surface sharp.

How to use: ~4-6 problems + a few cards daily; **pattern-first** (recognize the pattern → recall the template → code it); re-solve solved problems from memory weekly; log every miss as a card. **Triangulation targets:** *DP setup (state/transition/base), binary-search-on-the-answer, C++ move semantics/RAII, and STL container complexity.*

Ranked hard-topics map

1. **DP** — recognizing it + defining state/transition/base (the #1 stumbling block).
2. **Graphs** — choosing BFS/DFS/Dijkstra/union-find and implementing cleanly.
3. **C++ memory model** — pointers/references, RAII, smart pointers, move semantics.
4. **Binary search variants** — on arrays *and* on a monotonic predicate ("on the answer").
5. **STL fluency** — which container, and its complexity/iterator-invalidation.
6. **Complexity analysis** — time+space, amortized, and stating it out loud.

Anki deck (Q → A)

Deck A · C++ language essentials

- **Q:** Reference vs pointer? → **A:** A reference is an alias (can't be null, can't rebind, no arithmetic); a pointer holds an address (nullable, rebinding, supports arithmetic). Prefer references unless you need nullability/reseating.
- **Q:** What is RAII? → **A:** Resource Acquisition Is Initialization — tie a resource's lifetime to an object's scope so the destructor releases it automatically (no leaks, exception-safe).
- **Q:** `unique_ptr` vs `shared_ptr` vs `weak_ptr`? → **A:** `unique_ptr` = sole ownership (move-only, zero overhead); `shared_ptr` = reference-counted shared ownership; `weak_ptr` = non-owning observer that breaks `shared_ptr` cycles.
- **Q:** Move semantics / `std::move`? → **A:** Transfer resources from an rvalue instead of copying (steal the buffer); `std::move` is just a cast to an rvalue reference — it enables the move, doesn't move by itself.
- **Q:** Rule of 0/3/5? → **A:** Rule of 0: rely on defaults (use RAII members). If you define one of {destructor, copy-ctor, copy-assign} you likely need all three (Rule of 3); add move-ctor + move-assign for Rule of 5.
- **Q:** const correctness — why? → **A:** Marks what a function won't modify (const methods, const&) → compiler-enforced contracts, safer APIs, enables optimizations, and lets you pass temporaries.
- **Q:** lvalue vs rvalue? → **A:** lvalue = has a name/address (persists); rvalue = temporary (about to expire). `T&` binds rvalues → move semantics/perfect forwarding.
- **Q:** Pass by value vs const reference? → **A:** Pass big objects by `const T&` to avoid copies; pass small/trivially-copyable by value; pass by value + `std::move` when you'll store a copy anyway.
- **Q:** Why are templates header-only? → **A:** The compiler instantiates a template only when it sees the concrete type, so the full definition must be visible at the use site (in the header).
- **Q:** Two common undefined-behavior traps? → **A:** Out-of-bounds access / dangling reference/iterator; signed integer overflow; using a moved-from object beyond a valid but unspecified state.
- **Q:** `emplace_back` vs `push_back`? → **A:** `emplace_back` constructs in place from the args (can avoid a temporary/move); `push_back` takes an already-constructed object.
- **Q:** Why `reserve()` on a vector? → **A:** Pre-allocate capacity to avoid repeated reallocations (and iterator invalidation) during known-size growth.

Deck B · STL containers & complexity

- **Q:** vector — layout + key complexities? → **A:** Contiguous array; $O(1)$ index, $O(1)$ amortized `push_back` (doubling), $O(n)$ insert/erase in the middle; cache-friendly.
- **Q:** `unordered_map` vs `map`? → **A:** `unordered_map` = hash table, $O(1)$ average / $O(n)$ worst, unordered; `map` = balanced BST, $O(\log n)$, keys sorted (supports ordered traversal/range).
- **Q:** When to use `set/multiset`? → **A:** Ordered unique/duplicate keys with $O(\log n)$ insert/find and ordered iteration (e.g., running median, floor/ceil queries).
- **Q:** `priority_queue` — what is it? → **A:** A binary heap; $O(\log n)$ push/pop, $O(1)$ top; max-heap by default (use `greater<>` for min-heap).
- **Q:** deque — when? → **A:** $O(1)$ push/pop at both ends → sliding-window maximum (monotonic deque), BFS on 0/1 weights.
- **Q:** Iterator invalidation gotcha? → **A:** vector reallocation invalidates all iterators/pointers; erasing invalidates from the erase point; `unordered_map` rehash invalidates iterators (not references). Don't hold iterators across mutations.

Deck C · Core DSA & complexity

- **Q:** Big-O: time vs space vs amortized? → **A:** Time = operation count growth; space = extra memory; amortized = average per-op over a sequence (e.g., vector `push_back` is amortized $O(1)$).
- **Q:** Master theorem (one line)? → **A:** For $T(n)=aT(n/b)+f(n)$, compare $f(n)$ to $n^{\log_b a}$ to get the class (e.g., merge sort $a=2, b=2, f=n \rightarrow O(n \log n)$).
- **Q:** Two-pointer / sliding window — when? → **A:** Contiguous subarray/substring or sorted-pair problems → $O(n)$ instead of $O(n^2)$ (longest-without-repeat, min-window, pair-sum).
- **Q:** Binary search — the two flavors? → **A:** On a sorted array (find target), and **on the answer** (smallest/largest x satisfying a monotonic predicate $P(x)$) — e.g., "min capacity to ship in D days".
- **Q:** Detect a cycle in a linked list? → **A:** Floyd's tortoise-and-hare — slow/fast pointers meet iff there's a cycle; $O(1)$ space.
- **Q:** Monotonic stack — what for? → **A:** Next-greater/smaller element, largest rectangle in histogram, stock span → $O(n)$ by maintaining a

monotonic stack of indices.

- **Q:** BST property + in-order traversal? → **A:** Left < node < right for every node; in-order traversal yields sorted order.
- **Q:** Tree traversals — pre/in/post/level? → **A:** Pre = node,L,R; in = L,node,R; post = L,R,node (DFS variants); level-order = BFS with a queue.
- **Q:** Heap: build vs push/pop complexity? → **A:** Build-heap (heapify) = $O(n)$; push/pop = $O(\log n)$; peek = $O(1)$.
- **Q:** Trie — when? → **A:** Prefix queries / autocomplete / word-dictionary problems → $O(L)$ per operation in the word length L .

Deck D · Graphs

- **Q:** Adjacency list vs matrix? → **A:** List = $O(V+E)$ space, good for sparse; matrix = $O(V^2)$ space, $O(1)$ edge lookup, good for dense/edge-heavy queries.
- **Q:** BFS vs DFS — when? → **A:** BFS = shortest path in *unweighted* graphs, level-order; DFS = connectivity, cycle detection, topological sort, backtracking.
- **Q:** Dijkstra — assumption + complexity? → **A:** Non-negative edge weights; $O((V+E) \log V)$ with a min-heap; greedily finalizes the closest unvisited node.
- **Q:** When Bellman-Ford over Dijkstra? → **A:** Negative edge weights (detects negative cycles); $O(VE)$.
- **Q:** Topological sort — two methods? → **A:** Kahn's (repeatedly remove in-degree-0 nodes) or DFS finish-order; only valid on a DAG.
- **Q:** Union-Find — the two optimizations? → **A:** Path compression + union by rank/size → near- $O(1)$ (inverse-Ackermann) per op; used for connectivity and Kruskal's MST.
- **Q:** MST algorithms? → **A:** Kruskal (sort edges + union-find) and Prim (grow a tree with a heap); both $O(E \log V)$ -ish.

Deck E · Dynamic programming & greedy

- **Q:** How to *set up* a DP? → **A:** Confirm overlapping subproblems + optimal substructure → define the **state** (what parameters index a subproblem), the **transition** (recurrence), and the **base case**; then memoize or tabulate.
- **Q:** Top-down vs bottom-up DP? → **A:** Top-down = recursion + memoization (natural, only computes needed states); bottom-up = iterative table (no recursion overhead, easier to optimize space).
- **Q:** 0/1 vs unbounded knapsack — the loop difference? → **A:** 0/1: iterate capacity *descending* (each item once); unbounded: iterate *ascending* (item reusable).
- **Q:** LIS in $O(n \log n)$? → **A:** Maintain a "tails" array; binary-search the insertion point of each element → length = size of tails.
- **Q:** Edit distance / LCS — state? → **A:** $dp[i][j]$ over prefixes of the two strings; transitions from match/insert/delete (edit) or match/skip (LCS).
- **Q:** When does greedy work? → **A:** When a local optimum provably leads to a global one — justify with an exchange argument or a matroid; otherwise use DP.

Deck F · Practice discipline (the thread)

- **Q:** Daily cadence? → **A:** ~4–6 problems: new ones by pattern, plus re-solving a couple you've done — spaced, then timed.
- **Q:** How to review a missed problem? → **A:** Understand the pattern (not just the answer), write a one-line "trigger-technique" card, and re-solve it 3 days later from a blank editor.
- **Q:** The checkpoint bar? → **A:** A random LeetCode **medium in ~25–30 min, hard in ~40–45**, code running, complexity stated.

Common misconceptions & traps

- **"unordered_map is always faster than map."** Worst-case $O(n)$ (hash collisions/attacks) and worse cache behavior; for small/ordered/range needs, map can win.
- **"Recursion is always cleaner."** Deep recursion risks stack overflow and recomputation — memoize or convert to iteration.
- **"Optimize first."** Get a correct brute force + its complexity first; then optimize with a named pattern.
- **"vector<bool> is a normal vector."** It's a bit-packed specialization with proxy references — surprising behavior; use `deque<bool>/vector<char>` if you need real bools.
- **"auto copies are free."** `auto x = bigRef`; copies — use `auto&/const auto&` to avoid it.

Glossary starter

reference vs pointer · RAI1 · unique/shared/weak_ptr · move semantics / `std::move` · rule of 0/3/5 · const-correctness · lvalue/rvalue · templates (header-only) · UB · vector/reserve/emplace_back · unordered_map vs map · priority_queue/heap · deque · iterator invalidation · Big-O (time/space/amortized) · master theorem · two-pointer/sliding-window · binary-search-on-answer · Floyd cycle · monotonic stack · BST/traversals · trie · adjacency list/matrix · BFS/DFS · Dijkstra/Bellman-Ford · topological sort · union-find · MST (Kruskal/Prim) · DP (state/transition/base) · knapsack/LIS/edit-distance · greedy (exchange argument).

Drills

Whiteboard: derive a DP recurrence for edit distance/knapsack; explain binary-search-on-the-answer with an example; contrast BFS/DFS/Dijkstra/union-find selection; explain RAI1 + rule of 5. **Blank-file (C++, no AI, timed):** implement a min-heap; Dijkstra with a priority_queue; union-find with path compression; LIS in $O(n \log n)$; a monotonic-deque sliding-window max; reverse a linked list + Floyd cycle detection.

Leaving bar (checkpoint, not a one-time gate)

A random medium in ~25–30 min and a hard in ~40–45, from a blank editor, code running, complexity (time+space) stated aloud; core structures + standard patterns reproducible from memory in idiomatic C++.